

Arent Bridge SE Vietnam — Week 1 Reading Pack

Mentee: Hpp (Hung)

Audience: Web developer mạnh · chưa biết C# · hợp JP ổn · chưa CAD/Revit

Ngôn ngữ: tiếng Việt (giải thích) · English (identifiers/code) · 日本語用語 (BrSE)

Phạm vi: mentor pack nội bộ — không liên hệ Arent

Đồng hành: 02-aabb-cheat-sheet.pdf (in bàn khi code)

Mục lục

	Chương	Nội dung chính
A	Mục tiêu tuần 1 + checklist	Outcome, việc phải tick, việc <i>không</i> làm
B	C# for web devs — mental model map	Types, null, class/struct, collections, records, namespace
C	.NET 8 console + Rider tips	Solution anatomy, CLI, debug, hotkeys
D	Readable code habits	Naming, early return, magic numbers, small functions
E	AABB 3D	Định nghĩa, axis separation, JSON, ví dụ giao/không giao
F	Mini 用語 JP tuần 1	8 từ BrSE + gloss VN + câu mẫu
G	Exercise brief	Success criteria AABB console app
H	Reading links	Official MS Learn / .NET docs (đã verify URL)

A. Mục tiêu tuần 1 + checklist

A.1 Mục tiêu (outcome)

Cuối tuần 1 bạn có thể:

1. Mở **JetBrains Rider**, tạo và chạy **.NET 8 console** solution không cần hỏi từng bước CLI.
2. Đọc/skim **Microsoft Learn — Get started with C#** đủ để viết type, method, `List`, parse JSON.
3. Ship một app console: đọc **JSON danh sách AABB**, kiểm tra **intersection**, in cặp giao rõ ràng.
4. Trong hợp JP ngắn, dùng đúng vài 用語: 干渉, アドイン, 図面, 配管 / 配筋 (chưa cần Revit API).

A.2 Checklist

#	Việc	Done?
1	Cài .NET 8 SDK — <code>dotnet --list-sdks</code> thấy 8.x	☐
2	Cài JetBrains Rider; mở được <code>.sln</code>	☐
3	<code>dotnet new console -n AabbLab -f net8.0</code> → chạy Hello	☐
4	MS Learn path <i>Get started with C#, Part 1</i> — skim theo chương B	☐
5	Parse JSON → <code>List<Aabb></code> bằng <code>System.Text.Json</code>	☐
6	<code>Intersects(Aabb a, Aabb b)</code> đúng axis separation	☐
7	Fixture ≥ 5 case từ cheat-sheet; in cặp giao khớp expected	☐
8	Đọc hết pack này + bảng 用語 F	☐
9	README 5–10 dòng: schema JSON + cách chạy + ví dụ output	☐

A.3 Không cần tuần này

Revit / CAD · WPF / WinForms · ASP.NET · `async/await` sâu · LINQ query syntax nâng cao · spatial index (BVH/octree) · tolerance số học phức tạp · liên hệ Arent.

A.4 Cách dùng pack

- Đọc **A** → **B** → **C** trước khi gõ project.
- Để **E** + **cheat-sheet** mở khi implement `Intersects`.
- **D** đọc khi refactor lần đầu (sau khi “chạy được”).
- **F** đọc trước buổi sync JP trong tuần.
- **G** là Definition of Done — tự chấm trước khi báo mentor.
- **H** chỉ mở link khi kệt; không binge cả catalog.

B. C# for web devs — mental model map

Bạn đã quen JS/TS (hoặc Python/Java). Mục tiêu: map khái niệm, không học thuộc lòng BCL.

B.1 Types — bảng map nhanh

Ý niệm web / TS	C#	Ghi chú thực dụng
number	int, long, float, double, decimal	AABB dùng double là đủ tuần 1
string	string	Immutable; so sánh == theo giá trị nội dung
boolean	bool	true / false (lowercase)
null / undefined	null + nullable	string?, int? — bật Nullable trong csproj
object / interface	class, interface, record	
Array / T[]	T[], List<T>	Prefer List<T> khi thêm/bớt
Map / object KV	Dictionary<TKey, TValue>	Key phải hashable / equality rõ
Promise<T>	Task / Task<T>	FYI — chưa bắt buộc exercise
any	object / dynamic	Tránh dynamic trừ khi có lý do mạnh

```
int count = 3;
double lengthMm = 12.5;
bool overlaps = true;
string id = "pipe-01";
string? maybeName = null; // nullable reference
int? optionalIndex = 42; // Nullable<int>
```

B.2 null và nullable — khác JS chỗ nào?

- C# phân biệt **value type** (int, bool, struct) và **reference type** (class, string, array).
- Project mới thường bật **nullable reference types**: compiler cảnh báo khi có thể dereference null.
- T? với value type = Nullable<T> (có HasValue / Value).
- Pattern hay dùng: ??, ?. , is null, ArgumentNullException.ThrowIfNull.

```
string label = maybeName ?? "(unnamed)";
int len = maybeName?.Length ?? 0;

void Use(Aabb box)
{
    ArgumentNullException.ThrowIfNull(box);
    // ...
}
```

Lỗi web-dev hay mắc: coi mọi thứ như JS undefined “nuốt” được. Trong C#, NRE (NullReferenceException) crash rõ — tốt, nhưng phải phòng từ biên JSON.

B.3 class vs struct

	class	struct
Lưu trữ	Reference (heap)	Value (inline / stack thường gặp)
null mặc định	Có thể	Không (trừ T?)
Assignment	Copy reference	Copy field values
Equality mặc định	Reference	Field-by-field
Khi nào	Domain object, service, hầu hết code	Nhỏ, ít field, thường immutable (Point, Aabb)

```
public class BoxService { }           // reference

public readonly struct Point3
{
    public double X { get; }
    public double Y { get; }
    public double Z { get; }
    public Point3(double x, double y, double z) => (X, Y, Z) = (x, y, z);
}
```

Tuần 1: readonly record struct Aabb(...) hoặc class Aabb đều chấp nhận — ưu tiên **rõ ràng + test được**.

B.4 Collections: IEnumerable · List · Dictionary

```
IEnumerable<Aabb> Query(IEnumerable<Aabb> source)
{
    foreach (var box in source)
        if (box.MaxX > 0)
            yield return box;    // lazy
}

List<Aabb> materialize = Query(all).ToList();
Aabb first = materialize[0];
Dictionary<string, Aabb> byId = materialize.ToDictionary(b => b.Id);
bool has = byId.ContainsKey("pipe-01");
```

Type	Gần với	Khi dùng
IEnumerable<T>	iterable / “có thể foreach”	Tham số đầu vào linh hoạt
List<T>	Array JS có push	Cần index, Count, Add
Dictionary<K,V>	Map	Lookup theo id
ReadOnlyList<T>	freeze-ish view	API không cho caller mutate

B.5 record — DTO / value data

```
public readonly record struct Aabb(  
    string Id,  
    double MinX, double MinY, double MinZ,  
    double MaxX, double MaxY, double MaxZ)  
{  
    public bool IsValid => MinX <= MaxX && MinY <= MaxY && MinZ <= MaxZ;  
}
```

- Equality theo giá trị (tiện assert trong test).
- with expression nếu dùng record class (ít cần tuần 1).

B.6 Properties vs fields

```
public class Sample  
{  
    private readonly List<Aabb> _boxes = new();           // field  
    public IReadOnlyList<Aabb> Boxes => _boxes;          // get-only property  
    public string Name { get; init; } = "";               // init-only (object initializer)  
}
```

Web analogy: property $\approx$ public field có control (get/set) — C# khuyến khích property cho API công khai.

B.7 Methods, static, top-level statements

```
// Program.cs – top-level (giống script entry)  
var boxes = BoxLoader.Load("boxes.json");  
Console.WriteLine(boxes.Count);  
  
public static class Geometry  
{  
    public static bool Intersects(Aabb a, Aabb b) { /* ... */ }  
}
```

- static method $\approx$ không cần instance (như utility).
- Top-level statements chỉ trong một file entry — gọn cho console lab.

B.8 Namespaces & using

```
using System.Text.Json;  
using System.Text.Json.Serialization;  
  
namespace AabbLab;  
  
public static class BoxLoader { }
```

- using X; = import namespace (không import từng symbol như Python from).
- File-scoped namespace AabbLab; tránh indent thừa.
- global using (ImplicitUsings trong SDK) đã kéo System, System.Collections.Generic, System.Linq, ...

B.9 async — chỉ FYI

```
// Đủ biết tồn tại. Exercise tuần 1 dùng sync:
string json = File.ReadAllText(path);

// Sau này:
string jsonAsync = await File.ReadAllTextAsync(path);
```

Đừng refactor sang async chỉ vì “.NET hay async” — chưa có I/O song song / UI.

B.10 JS/TS → C# “gotchas” ngắn

JS/TS	C#
===	== / Equals / ReferenceEquals — biết mình so sánh gì
duck typing	compile-time types; generic <code>List<T></code>
<code>JSON.parse</code>	<code>JsonSerializer.Deserialize<T>(...)</code> cần model
<code>for..of</code>	<code>foreach</code>
optional chaining <code>?.</code>	có <code>?.</code> và <code>??</code>
default args	có; thêm overload khi API phức tạp

C. .NET 8 console project anatomy + Rider tips

C.1 Solution vs project

```
AabbLab.sln      ← Solution: “workspace” chứa 1..n project
├─ AabbLab/
│   ├─ AabbLab.csproj      ← Project: TFM, NuGet, compile items
│   ├─ Program.cs          ← Entry point
│   ├─ Geometry/
│   │   └─ Aabb.cs
│   ├─ Io/
│   │   └─ BoxJson.cs
│   └─ boxes.sample.json    ← Copy to output (xem dưới)
```

Khái niệm	File	Vai trò
Solution	<code>.sln</code>	Rider/VS mở cái này
Project	<code>.csproj</code>	Build unit; <code>TargetFramework</code>
Assembly	output <code>.dll</code> / <code>.exe</code>	Kết quả compile

C.2 Tạo từ CLI (nên làm một lần bằng tay)

```
dotnet new sln -n AabbLab
dotnet new console -n AabbLab -f net8.0
dotnet sln AabbLab.sln add AabbLab/AabbLab.csproj
cd AabbLab && dotnet run
```

Thêm test project (tuỳ chọn nhưng tốt):

```
dotnet new xunit -n AabbLab.Tests -f net8.0
dotnet sln add AabbLab.Tests/AabbLab.Tests.csproj
dotnet add AabbLab.Tests/AabbLab.Tests.csproj reference AabbLab/AabbLab.csproj
dotnet test
```

C.3 AabbLab.csproj — đọc được phần quan trọng

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <None Update="boxes.sample.json">
      <CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>
    </None>
  </ItemGroup>
</Project>
```

- **System.Text.Json** nằm trong BCL hiện đại — thường **không** cần package NuGet thêm cho parse JSON cơ bản.
- NuGet khi cần: Rider → NuGet; hoặc `dotnet add package Some.Package`.

C.4 Lệnh hay dùng

Lệnh	Việc
<code>dotnet build</code>	Compile
<code>dotnet run --project AabbLab</code>	Build + chạy
<code>dotnet run -- boxes.sample.json</code>	Truyền args sau <code>--</code>
<code>dotnet test</code>	Chạy test
<code>dotnet --list-sdks</code>	Xem SDK đã cài

C.5 Rider — workflow tối thiểu

Việc	Cách
Mở đúng	Open <code>.sln</code> , không chỉ folder lung tung
Run	▶ (toolbar) hoặc green arrow cạnh Main /top-level
Debug	Bug icon; breakpoint click lề trái dòng
Step	F8 step over · F7 step into · Shift+F8 step out (keymap mặc định JetBrains — kiểm tra Keymap của bạn)
Rename an toàn	Shift+F6
Find usages	Alt+F7
Go to type	Ctrl+N (hoặc Cmd+O trên macOS keymap)
Go to file	Ctrl+Shift+N
Format	Ctrl+Alt+L
Terminal	View → Tool Windows → Terminal
NuGet UI	Search package, Add to project

Debug tip AABB: breakpoint trong `Intersects`, xem `MinX / MaxX` từng nhánh early-return — nhanh hơn spam `Console.WriteLine`.

C.6 Cấu trúc thư mục gợi ý tuần 1

Giữ nông — đừng over-engineer:

```
Program.cs           // args, orchestration, print
Geometry/Aabb.cs     // type + Intersects (+ helpers tách trực nếu muốn)
Io/BoxLoader.cs      // DTO JSON + map sang Aabb + validate
boxes.sample.json
```

D. Readable code habits (*Art of Readable Code* × C#)

Nguồn cảm hứng: *The Art of Readable Code* (Dustin Boswell & Trevor Foucher). Không tóm tắt cả sách — chỉ thói quen áp dụng ngay cho lab.

D.1 Đặt tên nói “ý định”, không nói “cơ chế mơ hồ”

```
// Bad – đọc không biết trả về gì
bool F(Aabb a, Aabb b) => !(a.MaxX < b.MinX || b.MaxX < a.MinX || ...);

// Better
bool Intersects(Aabb a, Aabb b) => ...;

bool AreSeparatedOnX(Aabb a, Aabb b) =>
    a.MaxX < b.MinX || b.MaxX < a.MinX;
```

Tên biến: `boxes`, `left`, `right`, `pair` — tránh `temp`, `data2`, `obj`.

D.2 Early return — giảm nesting

```
public static bool Intersects(Aabb a, Aabb b)
{
    if (AreSeparatedOnX(a, b)) return false;
    if (AreSeparatedOnY(a, b)) return false;
    if (AreSeparatedOnZ(a, b)) return false;
    return true;
}
```

Tương đương một biểu thức `!(sepX || sepY || sepZ)` — cả hai đều ổn nếu **đúng** và **đọc một mạch**.

D.3 Không magic numbers

```
// Bad
if (boxes.Count > 1000) { /* ... */ }

// Better
const int MaxBoxesForNaivePairs = 1000;
if (boxes.Count > MaxBoxesForNaivePairs) { /* warn */ }
```

Tuần 1 so sánh `double` strict trên fixture cố định là chấp nhận được. Nếu sau này có tolerance:

```
const double Epsilon = 1e-9;
static bool Less(double a, double b) => a < b - Epsilon;
```

D.4 Hàm nhỏ — một việc

```
public static IEnumerable<(string A, string B)> FindIntersectingPairs(
    IReadOnlyList<Aabb> boxes)
{
    for (int i = 0; i < boxes.Count; i++)
    {
        for (int j = i + 1; j < boxes.Count; j++)
        {
            if (Intersects(boxes[i], boxes[j]))
                yield return (boxes[i].Id, boxes[j].Id);
        }
    }
}
```

Tách: load JSON ≠ geometry ≠ print. Để test `Intersects` độc lập.

D.5 Comment “why”, không “what”

```
// Separating-axis test: một trục tách hẳn ⇒ không giao (AABB broad-phase).
if (AreSeparatedOnX(a, b)) return false;
```

Tránh comment kiểu `// tăng i cạnh i++`.

D.6 Consistent style trong một file

- Một cách đặt brace / naming (`PascalCase` public, `camelCase` local).
- Đừng mix `MinX` và `min_x` trong cùng model.
- JSON property name: chọn `camelCase` **hoặc** map bằng `[JsonPropertyName]` — document trong README.


```
static bool AreSeparatedOnX(Aabb a, Aabb b) =>
    a.MaxX < b.MinX || b.MaxX < a.MinX;

static bool Intersects(Aabb a, Aabb b) =>
    !AreSeparatedOnX(a, b)
    && !AreSeparatedOnY(a, b)
    && !AreSeparatedOnZ(a, b);
```

E.3 Vì sao BrSE / Revit quan tâm? (干涉検出 mindset)

Trong BIM/MEP:

1. Model có hàng nghìn phần tử (配管, 配筋, duct, thiết bị...).
2. Kiểm tra hình học chi tiết mọi cặp = đắt.
3. **Broad-phase:** loại nhanh cặp có AABB **không** giao.
4. **Narrow-phase:** chỉ cặp nghi ngờ → soi solid/mesh chính xác hơn.
5. Kết quả báo 干涉 để 詳細設計 / phối hợp hiện trường.

BrSE không nhất thiết code hết pipeline — nhưng phải **hiểu** logic AABB để nói chuyện được với dev / JP reviewer: “干涉 này có phải false positive do box thô không?”

E.4 JSON schema thực hành

Schema đề xuất (nested min/max):

```
{
  "boxes": [
    {
      "id": "pipe-01",
      "min": { "x": 0.0, "y": 0.0, "z": 0.0 },
      "max": { "x": 2.0, "y": 1.0, "z": 1.0 }
    }
  ]
}
```

Phương án flat (cũng OK nếu README nói rõ):

```
{
  "id": "rebar-A",
  "minX": 1, "minY": 0, "minZ": 0,
  "maxX": 3, "maxY": 0.5, "maxZ": 2
}
```

Validate khi load:

Rule	Hành vi gợi ý
thiếu id / trùng id	Fail rõ message
min > max trên trục	Fail hoặc skip + warn
không phải số	Fail deserialize

DTO ví dụ:

```

public sealed class BoxFileDto
{
    public List<BoxDto> Boxes { get; set; } = new();
}

public sealed class BoxDto
{
    public string Id { get; set; } = "";
    public Vec3Dto Min { get; set; } = new();
    public Vec3Dto Max { get; set; } = new();
}

public sealed class Vec3Dto
{
    public double X { get; set; }
    public double Y { get; set; }
    public double Z { get; set; }
}

```

```

var options = new JsonSerializerOptions { PropertyNameCaseInsensitive = true };
var dto = JsonSerializer.Deserialize<BoxFileDto>(json, options)
    ?? throw new InvalidOperationException("Empty JSON");

```

E.5 Worked examples

Case	A min→max	B min→max	Kết quả	Lý do
1	(0,0,0)→(2,2,2)	(1,1,1)→(3,3,3)	Intersect	Chồng 3 trục
2	(0,0,0)→(1,1,1)	(2,0,0)→(3,1,1)	No	Tách X (1 < 2)
3	(0,0,0)→(1,1,1)	(1,0,0)→(2,1,1)	Intersect	Chạm mặt X
4	(0,0,0)→(5,1,1)	(1,2,0)→(2,3,1)	No	Tách Y
5	(0,0,0)→(1,1,1)	(0,0,2)→(1,1,3)	No	Tách Z
6	(0,0,0)→(1,1,1)	(0,0,0)→(1,1,1)	Intersect	Trùng khớp
7	(0,0,0)→(1,1,1)	(1,1,1)→(2,2,2)	Intersect	Chạm góc

Tự vẽ nhanh trên giấy 2D (bỏ Z) trước khi code — giảm bug off-by-one tư duy.

E.6 Độ phức tạp naive

So cặp n box: $O(n^2)$ phép `Intersects`. Với lab $n \leq$ vài chục là ổn. Production BIM dùng hierarchy không gian — **không** yêu cầu tuần 1.

F. Mini 用語 JP tuần 1

Dùng trong họp / Slack / review tài liệu BrSE. Chưa cần thuộc Revit API.

用語	Đọc (approx.)	Gloss VN	Ví dụ câu JP
アドイン	adoin	Plugin / extension (vd. Revit add-in)	この機能はRevitのアドインで実装します。
ファミリ	famirī	Family — định nghĩa loại đối tượng BIM	配管のファミリを確認してください。
配筋	haikin	Cốt thép / reinforcement	配筋とスリーブの干渉をチェックします。
干渉	kanshō	Clash / interference (va chạm hình học)	干渉が見つかりました。原因を切り分けましょう。
配管	haikan	Piping / ống hệ MEP	配管ルートを変更すれば干渉を回避できます。
図面	zumen	Drawing / bản vẽ	図面とモデルの整合を確認します。
スナップショット	sunappushotto	Snapshot — bản chụp trạng thái model/data	検証前にスナップショットを取ってください。
詳細設計	shōsai sekkei	Detailed design — thiết kế chi tiết	詳細設計レビューで干渉方針を合わせます。

F.1 Mẫu thoại hợp ngăn

- ・「AABBの段階で干渉候補を出しています。」
(*Đang xuất ứng viên 干渉 ở bước AABB.*)
- ・「この干渉は配管と配筋です。図面も確認します。」
(*干渉 này là ống và cốt thép. Cũng sẽ xem 図面.*)
- ・「アドインからスナップショットを保存できますか？」
(*Từ アドイン có lưu スナップショット được không?*)

F.2 Tip nhớ nhanh

Gắn từ vào **verb**: 干渉を検出する / 回避する / 報告する・図面を確認する・アドインで実装する.

G. Exercise brief — AABB console app

G.1 Mục tiêu bài tập

Một console app .NET 8 đọc JSON AABB và liệt kê các cặp **giao nhau**, đúng fixture, code đọc được.

G.2 Success criteria (Definition of Done)

#	Tiêu chí	Pass khi
1	Solution chạy	<code>dotnet run --project AabbLab</code> (hoặc Rider ►) exit 0 trên sample
2	Input JSON	Đường dẫn file qua arg hoặc <code>boxes.sample.json</code> cạnh output
3	Model	Mỗi box có Id + min/max 3 trục; validate <code>min ≤ max</code>
4	Geometry	<code>Intersects</code> đúng case T1–T7 (cheat-sheet / bảng E.5)
5	Output	In tổng số box + danh sách cặp (<code>idA</code> , <code>idB</code>) có giao (sorted ổn định càng tốt)
6	Readable	Tên rõ, early-return / tách trục, không magic rải rác
7	Evidence	<code>dotnet test</code> nhỏ hoặc README ghi expected pairs của sample
8	Docs	README: schema, lệnh chạy, ví dụ stdout ~5–15 dòng

G.3 Output mẫu (minh hoạ)

```
Loaded 6 boxes from boxes.sample.json
Intersecting pairs (2):
  A x B
  A x D
```

G.4 Gợi ý triển khai theo bước

1. Hard-code 2 box trong `Program.cs` → in `Intersects` — khóa công thức.
2. Đưa vào `Geometry/Aabb.cs`.
3. Thêm JSON loader + sample file.
4. Pair loop + print.
5. Thêm test/README expected.
6. Refactor theo chương D (đặt tên, tách hàm).

G.5 Ngoài phạm vi

Revit API · UI · REST · performance index · UI diff 図面 · liên hệ stakeholder Arent.

G.6 Tự review trước khi gửi mentor

- [] Đổi `MaxX < MinX` nhầm thành `<=` có làm sai case chạm mặt không? (hiểu quy ước)
- [] Id trùng / box invalid có message rõ không?
- [] Chạy lại fixture sau khi “refactor đẹp”?

H. Reading links (official)

Chỉ URL Microsoft Learn / docs đã xác minh. Đọc đúng mục đích tuần 1 — không mở lan.

Title	URL	Why tuần 1
Write your first code using C# (Get started with C#, Part 1)	https://learn.microsoft.com/en-us/training/paths/get-started-c-sharp-part-1/	Learning path có module ngắn — đủ cú pháp console
Interactive tutorials — A tour of C#	https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/tutorials/	Tour interactive nếu thích học bằng snippet chạy được
Hello world (Tour of C#)	https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/tutorials/hello-world	Bài đầu tour — warmup nhanh
.NET documentation (fundamentals hub)	https://learn.microsoft.com/en-us/dotnet/fundamentals/	Hub: SDK, runtime, libraries — bookmark
Introduction to .NET	https://learn.microsoft.com/en-us/dotnet/core/introduction	Một trang “.NET là gì” trước khi đào sâu
Get started with .NET	https://learn.microsoft.com/en-us/dotnet/core/get-started	Tạo/chạy app đầu với SDK
JSON serialization (System.Text.Json overview)	https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/overview	Deserialize <code>boxes.json</code> đúng cách

H.1 Thứ tự đọc đề xuất

1. Path **Get started with C#, Part 1** — skim, làm nhanh chỗ lạ so với web.
2. Khi viết loader → **System.Text.Json overview**.
3. Kẹt khái niệm runtime/SDK → **Introduction to .NET + fundamentals hub**.
4. Tour interactive — optional.

Phụ lục — Nhật ký học gợi ý (không bắt buộc)

Ngày	Focus	Output
1	SDK + Rider + Hello	Checklist 1-3
2	MS Learn skim + types B	Ghi chú ½ trang gotchas
3	<code>Intersects</code> hard-coded	Pass T1-T3
4	JSON + pairs	Sample chạy
5	Test/README + 用語 F	DoD G.2 tick hết

Kết tuần 1

Bạn xong tuần 1 khi: **Rider + console chạy**, **Intersects đúng fixture**, README ngắn có expected pairs, và nói được 干渉 / アドイン / 図面 / 配管・配筋 trong ngữ cảnh BrSE.

Pack này là nền mentoring — không phải đề thi, không cam kết kết quả tuyển dụng.

Mentor reading pack · Week 1 · Arent Bridge SE Vietnam track · kèm 02-aabb-cheat-sheet.pdf